

Fondamenti di Informatica

A.A. 2017-2018

Prova intermedia del 4 luglio 2018 – Testo 1

NOME _____ COGNOME _____

Durante l'esame non si possono usare appunti né altro materiale.

Esercizio 1

Si dica se ciascuna delle seguenti affermazioni relative ai *template di funzione* in C++ è corretta:

Sì No

- | | | |
|--------------------------|--------------------------|--|
| ☐ | ☐ | In un template di funzione uno stesso parametro può assumere differenti tipi. |
| ☐ | ☐ | I template di funzione non possono avere più di un tipo generico. |
| ☐ | ☐ | Nei template di funzione il tipo che sostituisce il tipo generico può essere solo un tipo fondamentale. |
| ☐ | ☐ | I template di funzione permettono di definire un modello di funzione applicabile a diversi tipi di dato. |
| ☐ | ☐ | Nella chiamata di un template di funzione il tipo da utilizzare deve essere sempre esplicitato. |
| ☐ | ☐ | Quando il compilatore analizza la chiamata a un template di funzione, produce un codice in cui il tipo generico viene sostituito dal tipo effettivamente utilizzato. |
| ☐ | ☐ | La sostituzione del tipo generico con quello utilizzato avviene sempre in fase di esecuzione. |
| ☐ | ☐ | Nella dichiarazione di un template di funzione, la parola chiave <code>typename</code> introduce un tipo generico. |
| ☐ | ☐ | Nella dichiarazione di un template di funzione, al posto di <code>typename</code> non si può mai trovare <code>class</code> . |
| ☐ | ☐ | Le operazioni su variabili del tipo generico devono essere definite per il tipo effettivamente utilizzato. |

Esercizio 2

Completare le seguenti affermazioni:

1. Una variabile del tipo definito dalla classe è detta _____.
2. La rappresentazione (implementazione) di un tipo di dato astratto è detta _____.
3. In una lista concatenata, l'_____ di nuovi nodi può avvenire in testa, in coda o in mezzo alla lista.
4. Dato un puntatore a un oggetto, l'accesso ai suoi metodi avviene attraverso l'operatore _____.

Esercizio 3

Identificare e correggere gli errori nella seguente dichiarazione di classe C++:

```
1    class Test {
2    public:
3        Test();
4        void Test(int a);
5        Test(const Test& t);
6        int getX() const;
7        void setX(int a);
8        const Test& operator=(const Test& t);
9        ostream& operator<<(ostream& os, const Test& t);
10   private:
11       int _x;
12   };
```

Soluzione:

Esercizio 4

Si scriva il valore assunto dalle variabili a e b dopo l'esecuzione delle seguenti righe di codice:

Attenzione: è richiesto di riportare brevemente i passaggi seguiti per ottenere la soluzione.

```
1 1  int x = 15; int y = 10;      2 1  char b = 'R'; char s = 'S';
  2  int* px = &x;                2  char& q = b;
  3  int* py = &y;                3  char* ps = &s;
  4  *px = y + *py;              4  (*ps)++;
  5  int a = (*px + y);          5  b = 'A' + (*ps - q);
```

Soluzione:

Il seguente esercizio dovrà essere svolto al calcolatore.

Esercizio 5

- Per eseguire operazioni su un sottoinsieme di elementi di un array, si sviluppi in C++ la classe *ArrayOperations* avente i seguenti attributi: il puntatore *_pVal* a un array di numeri reali sul quale si eseguiranno le operazioni, la sua dimensione *_n* (un numero intero), il puntatore *_pMask* a un array di numeri interi di dimensione *_n*. Gli elementi dell'array puntato da *_pMask* possono assumere soltanto il valore 0 oppure 1. Per ciascun elemento dell'array puntato da *_pMask* di valore 1, il corrispondente elemento dell'array puntato da *_pVal* verrà considerato nel calcolo dell'operazione. Ad esempio, nel caso in cui l'operazione consista nella somma degli elementi dell'array, verranno sommati tra loro solo quegli elementi dell'array puntato da *_pVal* per i quali il corrispondente elemento dell'array puntato da *_pMask* vale 1. Si implementino, inoltre, i seguenti metodi:
 - Il costruttore di default che inizializzi *_n* a zero e i puntatori *_pVal* e *_pMask* a NULL.
 - Un costruttore con parametri che riceva un array *v* di numeri reali, un array *m* di numeri interi e la loro dimensione *dim* (un numero intero). Il costruttore opererà come segue: effettuerà l'allocazione dinamica di un array di *dim* numeri reali e ne assegnerà il puntatore a *_pVal*; effettuerà l'allocazione dinamica di un array di *dim* numeri interi e ne assegnerà il puntatore a *_pMask*; se l'allocazione dinamica della memoria va a buon fine in entrambi i casi, copierà i valori contenuti in *v* nell'array puntato da *_pVal*, copierà i valori contenuti in *m* nell'array puntato da *_pMask* e assegnerà *dim* a *_n*; nel caso in cui l'allocazione dinamica della memoria non vada a buon fine, il costruttore inizierà *_pVal* a NULL, *_pMask* a NULL e *_n* a zero.
 - Il costruttore di copia.
 - Il distruttore.
 - Un selettore per ciascuno dei tre attributi.
 - L'operatore di assegnamento.
 - Il metodo *calcola* che riceva come parametro il puntatore *pf* a una funzione che riceva come parametri un array di numeri reali, un array di numeri interi e la loro dimensione (un numero intero) e restituisca come valore di ritorno un numero reale. Il metodo chiamerà la funzione il cui puntatore *pf* è passato come parametro, passandole gli array puntati da *_pVal* e *_pMask* e la loro dimensione *_n* e restituirà il valore di ritorno restituito dalla funzione (un numero reale). Nel caso in cui *_pVal* o *_pMask* valgano NULL, il metodo ritornerà -1.
- Si sviluppino, quindi, le seguenti funzioni in linguaggio C++:
 - La funzione *somma* che riceva come parametri un array *v* di numeri reali, un array *m* di numeri interi e la loro dimensione *dim* (un numero intero) e restituisca come valore di ritorno un numero reale. La funzione calcolerà la somma degli elementi di *v* per i quali il corrispondente elemento di *m* vale 1.
 - La funzione *media* che riceva come parametri un array *v* di numeri reali, un array *m* di numeri interi e la loro dimensione *dim* (un numero intero) e restituisca come valore di ritorno un numero reale. La funzione calcolerà la media degli elementi di *v* per i quali il corrispondente elemento di *m* vale 1.
 - La funzione *main* che operi come segue: dichiarare un oggetto *q* di tipo *ArrayOperations*, utilizzando il costruttore con parametri cui vengano passati l'array di numeri reali $x = \{1.0, 2.0, 3.0\}$, l'array di numeri interi $y = \{1, 0, 1\}$ e la loro dimensione $dim = 3$; chiamare sull'oggetto *q* il metodo *calcola* della classe *ArrayOperations* passando come parametro per *pf* la funzione *somma* e stampare a video il risultato; chiamare sull'oggetto *q* il metodo *calcola* della classe *ArrayOperations* passando come parametro per *pf* la funzione *media* e stampare a video il risultato.

Soluzione

Esercizio 1 – Soluzione (1.5 punti, 0.15 per ogni risposta esatta)

Sì, No, No, Sì, No, Sì, No, Sì, No, Sì.

Esercizio 2 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

Oggetto, tipo concreto, inserimento, ->.

Esercizio 3 – Soluzione (1.0 punto, 0.5 per ogni risposta esatta)

Riga 4: il costruttore non ha valore di ritorno: void Test(int a); -> **Test(int a);**

Riga 9: l'operatore di inserimento è generalmente implementato come funzione amica

```
friend ostream& operator<<(ostream& os, const Test& t);
```

Esercizio 4 – Soluzione (1.5 punti, 0.75 per ogni risposta esatta)

a = 30, b = 'C'.

Esercizio 5 – Soluzione (6.0 punti, 1.0 – 0.25 – 0.5 – 0.5 – 0.25 – 0.25 – 0.25 – 0.25 – 0.5 – 0.75 – 0.5 – 0.5 – 0.5)

```
#include <iostream>
using namespace std;

class ArrayOp {
public:
    ArrayOp();
    ArrayOp(double v[], int m[], int dim);
    ArrayOp(const ArrayOp& ao);
    ~ArrayOp();
    const double* GetVals() const;
    const int* GetMask() const;
    int GetSize() const;
    const ArrayOp& operator= (const ArrayOp& ao);
    double calcola(double (*pf)(double[], int[], int));
private:
    double* _pVal;
    int* _pMask;
    int _n;
};

ArrayOp::ArrayOp() {
    _pVal = NULL; _pMask = NULL; _n = 0;
}

ArrayOp::ArrayOp(double v[], int m[], int dim) {
    _pVal = new double[dim];
    if (_pVal == NULL) {
        _pMask = NULL; _n = 0;
        return;
    }
    _pMask = new int[dim];
    if (_pMask == NULL) {
        delete[] _pVal;
        _pVal = NULL; _n = 0;
        return;
    }
    for (int k = 0; k < dim; k++) {
        _pVal[k] = v[k];
        _pMask[k] = m[k];
    }
    _n = dim;
}

ArrayOp::ArrayOp(const ArrayOp& ao) {
    _pVal = new double[ao._n];
    if (_pVal == NULL) {
        _pMask = NULL; _n = 0;
        return;
    }
    _pMask = new int[ao._n];
    if (_pMask == NULL) {
        delete[] _pVal;
        _pVal = NULL; _n = 0;
        return;
    }
    for (int k = 0; k < _n; k++) {
        _pVal[k] = ao._pVal[k];
        _pMask[k] = ao._pMask[k];
    }
    _n = ao._n;
}

ArrayOp::~ArrayOp() {
    if (_pVal != NULL) delete[] _pVal;
    if (_pMask != NULL) delete[] _pMask;
}

const double* ArrayOp::GetVals() const {
    return _pVal;
}

const int* ArrayOp::GetMask() const {
    return _pMask;
}

int ArrayOp::GetSize() const {
    return _n;
}

const ArrayOp& ArrayOp::operator=
    (const ArrayOp& ao) {
    if (this != &ao) {
        if (_pVal != NULL)
            delete[] _pVal;
        if (_pMask != NULL)
            delete[] _pMask;
        _pVal = new double[ao._n];
        if (_pVal == NULL) {
            _pMask = NULL; _n = 0;
            return *this;
        }
        _pMask = new int[ao._n];
        if (_pMask == NULL) {
            delete[] _pVal;
            _pVal = NULL; _n = 0;
            return *this;
        }
        for (int k = 0; k < _n; k++) {
            _pVal[k] = ao._pVal[k];
            _pMask[k] = ao._pMask[k];
        }
        _n = ao._n;
    }
    return *this;
}

double ArrayOp::calcola(double
    (*pf)(double[], int[], int)) {
    return pf(_pVal, _pMask, _n);
}

double somma(double a[], int m[], int dim) {
    double s = 0.0;
    for (int k = 0; k < dim; k++)
        if (m[k] != 0)
            s += a[k];
    return s;
}

double media(double a[], int m[], int dim) {
    double s = 0.0;
    int c = 0;
    for (int k = 0; k < dim; k++)
        if (m[k] != 0) {
            s += a[k];
            c++;
        }
    return (s / c);
}

int main() {
    const int d = 3;
    double x[d] = {1.0, 2.0, 3.0};
    int y[d] = {1, 0, 1};
    ArrayOp q(x, y, d);
    cout << q.calcola(somma) << endl;
    cout << q.calcola(media) << endl;
    return 0;
}
```